Multimodal Technologies and Interaction

MDPI

Article

A Digital Model-Based Serious Game for PID-Controller Education: One-Axis Drone Model, Analytics, and Student Study

Raul Brumar [1,*], Stelian Nicola [1] and Horia Ciocârlie [2]

[1] Department of Automation and Applied Informatics, Polytechnic University of Timisoara, 300006 Timisoara, Romania; stelian.nicola@upt.ro
[2] Department of Computer and Information Technology, Polytechnic University of Timisoara, 300006 Timisoara, Romania; horia.ciocarlie@cs.upt.ro
* Correspondence: raul.brumar@upt.ro

Abstract

This paper presents a serious game designed to support the teaching of PID controllers. The game couples a visually clear Unity scene with a physics-accurate digital model of a drone with a single degree of freedom (called a one-axis drone) and helps prepare students to meet the demands of Industry 4.0 and 5.0. An analytics back-end logs system error at 10 Hz and interaction metrics, enabling instructors to diagnose common tuning issues from a plot and to provide actionable hints to students. The design process that led to choosing the one-axis drone and turbulence application via "turbulence balls" is explained, after which the implementation is described. The proposed solution is evaluated in a within-subjects study performed with 21 students from mixed technical backgrounds across two short, unsupervised tinkering sessions of up to 10 min framed by four quizzes of both general and theoretical content. Three questions shaped the analysis: (i) whether error traces can be visualized by instructors to generate actionable hints for students; (ii) whether brief, unsupervised play sessions yield measurable gains in knowledge or stability; and (iii) whether efficiency of tuning improves without measurable changes in tune performance. Results show that analysis of plotted error values exposes recognizable issues with PID tunes that map to concrete hints provided by the instructor. When it comes to unsupervised play sessions, no systematic pre/post improvement in quiz scores or normalized area under absolute error was observed. However, it required significantly less effort from students in the second session to reach the same tune performance, indicating improved tuning efficiency. Overall, the proposed serious game with the digital twin-inspired one-axis drone and custom analytics back-end has emerged as a practical, safe, and low-cost auxiliary tool for teaching PID controllers, helping bridge the gap between theory and practice.

Keywords: serious game; digital games; digital game-based learning; digital model; digital twin; gamification; automation; regulator

Academic Editors: Stephan Schlögl, Ana Manzano-León and José M. Rodríguez Ferrer

Received: 3 September 2025
Revised: 9 October 2025
Accepted: 21 October 2025
Published: 24 October 2025

Citation: Brumar, R.; Nicola, S.; Ciocârlie, H. A Digital Model-Based Serious Game for PID-Controller Education: One-Axis Drone Model, Analytics, and Student Study. *Multimodal Technol. Interact.* **2025**, *9*, 111. https://doi.org/10.3390/mti9110111

1. Introduction

In the current educational context, technology plays an important role in transforming both teaching and learning. Integrating it into the teaching process offers opportunities to deepen the understanding of both theoretical and practical processes [1–4]. Technology has vast advantages, such as offering unlimited access to resources, personalized content through dedicated platforms, and increased engagement through interactive serious games.

The games can work using traditional technologies, or they can put players in a virtual or augmented reality to increase interactivity even further.

While theoretical knowledge is fundamental, lacking a clear grasp of how it is applied in practice defeats the purpose of learning. Theory gains real value when it can be applied in practice [5], and education should prepare students for the everyday challenges of the labor market [6,7]. Students should develop the required skills by applying their knowledge in concrete settings.

Technology allows us to develop innovative ways to introduce students to practical examples such as simulations and virtual laboratories. Problem-based projects, serious games situated in VR/AR, online collaboration platforms, and industry partnerships are some of the ways technology enables students to improve their practical skills [8,9].

Serious educational games, along with gamification, have become powerful tools for improving learning [4,10]. Although they are both inspired by traditional entertainment games, they represent two distinct approaches. Serious games are, in fact, games. However, in contrast to entertainment games, they are designed primarily for education, training, simulation, and problem solving. They integrate game mechanics into specific contexts so that learners can acquire knowledge through experience and direct interaction [11,12]. Gamification, on the other hand, is a design approach used in application development, not games, that applies elements and techniques with the goal of increasing user engagement, retention, and motivation [4,10]. In education, gamification reframes learning tasks as game-like progression [13,14].

The digital twin is an increasingly present concept when real-time virtual representations of a physical objects are discussed [15–17]. Kritzinger et al. [18] define a digital twin as a virtual counterpart that maintains bidirectional, real-time synchronization with its physical twin and distinguish it from simpler digital models or digital shadows. Jones et al. [19] expand this definition by introducing the concept of twinning rate, which uses the direction and frequency of the data flow to differentiate between digital twins in the literature. Still, as previous papers show, the term is often used in contexts where real-time coupling is not required. In such cases, which have started to outnumber the ones following the strict definition, the term digital twin-inspired simulation should be used to avoid confusing the field with the broader educational implementation of virtual laboratories.

The Proportional-Integral-Derivative (PID) controller is one of the most widely used algorithms due to its simplicity and applicability across different scenarios. Leveraging modern approaches such as digital twins and serious games opens new perspectives for teaching PID controllers [20,21].

This paper presents a serious game containing a digital twin-inspired simulation that has been designed to enhance the teaching of PID controllers. The game is built around a physically accurate digital model of a one-axis drone that is presented in a visually clear Unity scene. An analytics back-end records the error value at 10 Hz, which allows instructors to visualize it and provide targeted feedback to students. This paper documents the design and implementation steps taken for developing this serious game and analyzes the data obtained by testing it on a group of 21 students with diverse cultural and technical backgrounds. Both confirmatory and exploratory analyses were performed for answering the following three research questions:

- whether collected error values are useful for diagnosing students' issues with tuning;
- whether brief, unsupervised use of the serious game leads to measurable gains in student knowledge;
- whether a second attempt reduces tuning effort without degrading tune quality.

Overall, the contribution proved to be a safe, low-cost, and reliable teaching aid that complements traditional teaching methods and helps bridge the gap between theory and practice.

The paper is organized into five sections and contains twelve Figures and four Tables. Section 1 briefly presents the concepts underlying this study. Section 2 reviews related work and offers a short comparison across existing approaches. Section 3 examines the design and development of the proposed serious game. Section 4 reports the evaluation of quiz and analytics data collected during the student study. Section 5 summarizes the findings and outlines directions for future development.

2. Related Work

As discussed in the previous chapter, digital education requires approaches that are centered on the human factor, in a way that empowers human work. Industry 4.0 and 5.0 place digital education among the top priorities in terms of their approaches [22,23]. For this, the recent scientific literature highlights three major concepts: serious games, used to increase engagement through interactivity and active learning; digital twins, which offer a simulation faithful to reality on which students can experiment; and automation and controls, which are fundamental domains of industry [24,25]. These domains should not evolve separately. Instead, they should complement each other, shaping the education to be based on applications and serious games relevant to industrial concepts. This chapter examines the literature, split in three subsections:

- Serious Games in Education;
- Digital Twins in Education and Industry;
- Automation and Control in Industry 4.0/5.0.

2.1. Serious Games in Education

In recent studies, serious games play an important role in developing the competencies required by the digital industry. Pacheco-Velázquez et al. show that educational simulators can support training in logistics for Industry 4.0 by facilitating the understanding of complex processes in an interactive environment [26]. A systematic analysis performed in [27] indicates that the use of serious games in higher education has high potential within the Education 4.0 context. It also outlines key challenges related to development costs, integration in the curriculum, and lack of standardized evaluation methods. An automated evaluation system applied in serious VR games is presented in [28], where automated assessments were deemed to be comparable to human ones. Bertozzi et al. present an experimental study in industrial engineering education in which a serious game that has been integrated into a traditional course improved motivation and understanding of technical concepts [29]. Rodriguez-Calzada et al. report that the integration of serious games with active pedagogical methods increases engagement and student involvement. In a comparative study, agile learning through VR serious games outperformed the LEGO Serious Play method in teaching Scrum [30,31].

A subsection of serious games is represented by simulation-centric games, which emphasize realistic system behavior over having traditional win states. A review performed by [32] shows that this form of serious games is especially widespread in engineering education. Similar to Pacheco-Velázquez et al., a paper by Sánchez et al. presents three simulation-based educational games for teaching automatic control concepts [33].

In serious games, the main focus rests on motivation and engagement. However, realism is another element that is essential for a relevant educational environment. This role is fulfilled by the digital twin, a technology that brings learning closer to real-life applications and industrial contexts.

2.2. Digital Twin in Education and Industry

The industry uses digital twins especially for educating or training operators so that they better understand processes. Ortiz et al. [34] developed a digital twin for the system that processes cocoa beans. When used as an educational resource, users gained a better understanding of the process, significantly reducing operating times. Another digital twin increased motivation by facilitating access to virtual laboratories and involved reduced costs [35]. Many serious games are powered by digital twins, through which the student's state and progress can be tracked [36]. Zhang et al. report a quasi-experimental study with a learning system based on digital twins used for teaching landscape architecture courses. Students who use the system report a significant improvement in critical thinking, academic performance, and learning experience [37].

Another field where digital twins are present is medicine. Rajamäki et al. provide a review of the use of digital twins in higher medical education. The study highlights the advantages offered by simulating, diagnosing, and training in a controlled environment [38]. A recent concept called Digital Twin-Assisted Surgery offers digital replicas of the patient's anatomy, thus improving situational awareness and safety in complex scenarios [39]. Likewise, in oncology, digital twins are used for personalized therapeutic modeling. A systematic review presented applications for diagnosis, treatment decisions, prognosis prediction, and surgical planning by integrating medical imaging, multi-omics data, and AI techniques [40].

Digital twins provide realistic simulations and are very valuable when applied in fundamental technical fields. A good example field is automation and control, where digital twins integrated in serious games of virtual laboratories, paired with modern teaching methods, support adapting education to the requirements of Industry 4.0 and 5.0.

2.3. Automation and Control in Industry 4.0 and 5.0

If Industry 4.0 focused on digitalization and interconnection, Industry 5.0 emphasizes the human factor along with sustainability. Teaching these concepts requires adaptation to new technological paradigms. Visioli highlights the need to modernize control courses to include IoT, machine learning, and cyber–physical systems [41]. Azofeifa et al. explore the technological integrations present in Industry 4.0 and how they are used in the education of engineers. They propose competence visualization (KSA) and adaptive learning as ways to adapt to current challenges [42]. Ahmad et al. emphasize the benefits that Industry 4.0 technologies such as AI and robotics can have when applied to educational settings. This reinforces the link between teaching automation and new technologies [43]. While the PID controller is a fundamental part of automation, Industry 4.0 leverages it to support self-optimizing processes and predictive maintenance [44]. Dapkute and colleagues present the integration of traditional PID controllers with digital twin technologies in industrial process control. They propose a solution that can enable adaptive tuning and predictive maintenance through real-time simulations [45]. Even additive manufacturing can benefit from such an approach. There, the use of model predictive control (MPC) supported by digital twins and multivariable neural networks has been demonstrated to facilitate real-time decisions related to the manufacturing process [46].

The scientific literature shows that serious games, digital twins, and automatic control do not represent separate domains that are not linked, but are, instead, complementary parts of a modern educational ecosystem. Together, they create an educational approach in which theory, practice, and technology are used together, thus facilitating the development of competencies with direct effects in the industry.

3. Proposed Solution

The solution takes the form of a serious game designed to be used as an auxiliary tool when teaching PID controllers. The game should be designed to contain a digital twin-like physical system in which a PID controller is used to regulate a specific parameter. By providing a visual representation of the real-world system, along with interactive elements, the game aims to increase engagement and improve conceptual understanding of PID controllers. Players should adjust controller parameters (proportional, integral, and derivative coefficients) and, through real-time feedback, offered both in the form of text (system stats) and visually (graphics rendered by the game engine), immediately observe resulting changes in system behavior. This proposed solution not only reinforces theoretical concepts but also bridges the gap between abstract control theory and its practical implementation in engineering systems.

3.1. Choosing a Physical Twin

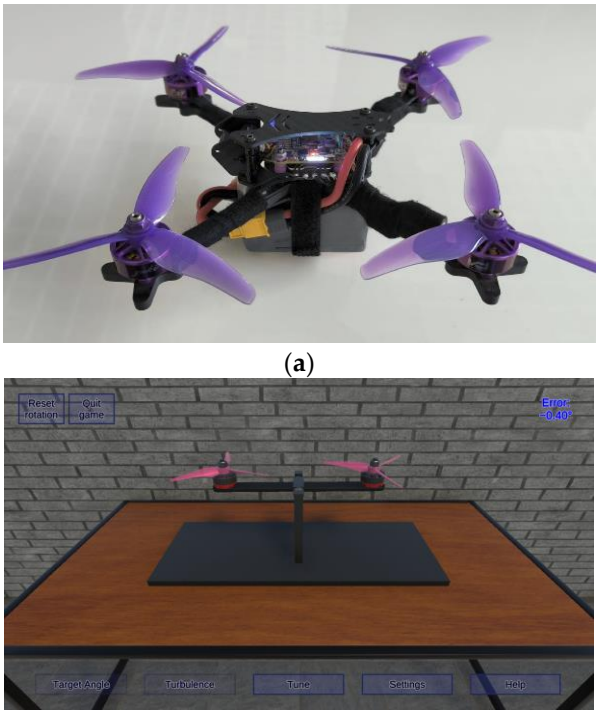
Before developing the game, a choice had to be made. Namely, which system was going to serve as the physical inspiration for the digital model used by the above-mentioned teaching process. There are a plethora of simple systems that could have been chosen, from cruise controls (traditional or adaptive) to temperature control or water level control in a tank. While the inner workings of a drone are technically harder to understand than other examples, its applicability in real-life is instantly recognizable, and the effects of tuning the controller are visually apparent. For these reasons, a drone was selected to teach students how a PID controller works. However, in order to reduce system complexity and to require a single set of tuning parameters, all but one degree of freedom was dropped, resulting in a simplified drone with a single degree of freedom. The remaining degree of freedom represents the roll axis, and can be influenced by adjusting the speed of the two remaining motors. This simplified one-axis drone serves as the real-world counterpart, or physical twin.

In Figure 1, we can observe the differences between a normal, hobby-grade drone with four motors (quadcopter—(a)) and the digital model of our simplified one-axis drone (b). While a quadcopter has four brushless motors, providing it with six degrees of freedom, our proposed model contains only two motors. On top of this, the one-axis drone is fixed on all but the roll angle, leaving it with a single degree of freedom. The biggest advantage of simplifying the physical twin is reducing the number of degrees of freedom, thus requiring PID-controller tuning for a single axis. Removing the yaw axis was especially important for two reasons: firstly, tuning the yaw axis is not as intuitive, as it is dependent on motor torque, and, secondly, implementing yaw control would require calculating the reaction torque of motors, which increases implementation complexity.

Some drawbacks to simplifying the physical twin are apparent, such as the loss of realism compared to a fully functional quadcopter and the omission of interactions between multiple degrees of freedom, but the advantages gained certainly outweigh these limitations in the context of this paper. While the reduced model cannot fully replicate the aerodynamic effects and control challenges present in its real-world counterpart, this level of fidelity is not essential for the primary objective of the project, which is to facilitate the understanding of PID-controller principles in a safe, interactive format. By limiting the system to a single degree of freedom, the cognitive burden on the students is reduced, helping them better interpret the relationship between the PID coefficients' adjustments and the system's response.

A digital twin-inspired simulation of the simplified one-axis drone has been chosen over a physical prototype for multiple reasons. The most important being that a digital implementation eliminates the costs associated with hardware, maintenance, and repairs.

Aside from the initial time investment, the project is freely replicable and accessible to as many students as needed. Another advantage comes from the removal of safety concerns that would arise from working with high-powered brushless motors and propellers. Such safety concerns range from equipment damage to personal injuries. Avoiding this means a safer and more productive learning environment. Lastly, the reduced complexity of a virtual system means that experiments will be easier to replicate consistently, without being influenced by the variability or malfunctions that physical hardware can introduce.



(**a**)

(**b**)

Figure 1. (**a**) The author's real-life racing quadcopter and (**b**) the digital model of a hypothetical one-axis drone.

3.2. Core Functionality

The serious game was designed around a limited number of core functions that the player can perform, with the goal of enabling experimentation with the PID controller in a safe environment. The feature that sits at the center of the game is the digital twin-inspired model of a simplified one-axis drone, which is simulated in real-time to respond to external disturbances based on the tuning received by the player.

Figure 2 presents the main actions undertaken by the student when playing the game. The process of tuning the digital model starts by observing the drone's physical state and is followed by tuning the parameters. After this, players can keep observing and tuning the parameters as long as they want, until a good tune is reached. After a good tune for counteracting gravity is reached, the player can impart disturbances through the use of balls falling from above the drone and tune the system accordingly. The described loop can then be performed until a satisfactory tune is reached. On top of this, the player can, at any point, slightly de-tune the system to observe how a bad tune would manifest on a physical twin.

Another complementary feature is present, namely the ability to modify the target roll angle. It can be set to either a fixed value, or to a slowly oscillating signal. Together with the ability to apply disturbances, these features provide a learning environment in which

students can iteratively refine the tune of the system, thereby increasing understanding of the PID controller.

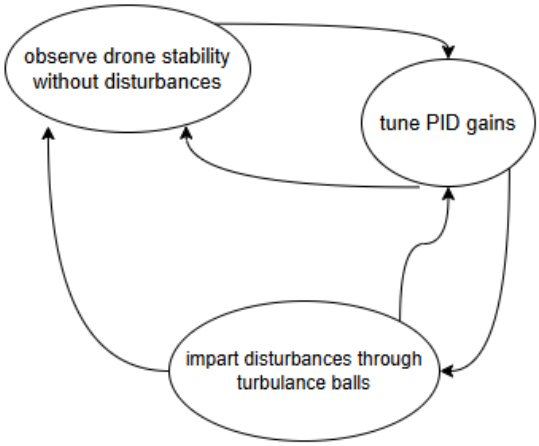


Figure 2. Core gameplay loop.

3.3. Implementation

The serious game was implemented in Unity 6000.1.10f1, ensuring a visually engaging environment. The physics simulation was carefully set up so that the drone's dynamics would closely resemble the physical twin in order to better support the intended learning outcomes.

3.3.1. Technologies Used

Most of the development was carried out in the Unity game engine as it provides high-fidelity rendering and physics simulation. The PID controller, other drone logic, and supporting functionalities were implemented in C# scripts attached to *GameObjects*. Blender 4.4.3 was used for modeling and adapting 3D models used in the scene. For version control, Git was used to manage the whole project [47].

3.3.2. Scene Setup

The main scene was designed to provide a realistic-looking and intuitive environment in which the digital model would be found. Aside from the simulated drone, the environment consists of a wooden table with metal legs situated on a tiled floor and in front of a grey brick wall. Another visually significant object is the metallic drone support. The support is not physics-simulated as it only serves to make it feel like the drone is fixed there, instead of floating.

At the core of the serious game lies the digital model of the simplified one-axis drone, represented in Unity as a parent *GameObject* called *OneAxisDrone* with multiple children. The parent contains the physics components, while the child *GameObjects* mostly ensure visual accuracy (motors, props, and drone body). Each motor is divided into two parts so that only the correct part rotates. The propeller is attached to the part that rotates.

Figure 3 illustrates the modular structure of the one-axis drone. The parent *GameObject* with its attached components handle physics and control of the drone, while the children are for purely aesthetic purposes, either static relative to the *OneAxisDrone* (*MRBody*, *Pivot*, *FrontBearing*, *BackBearing*, and stators), or dynamic (rotors, along with their respective props and nuts).

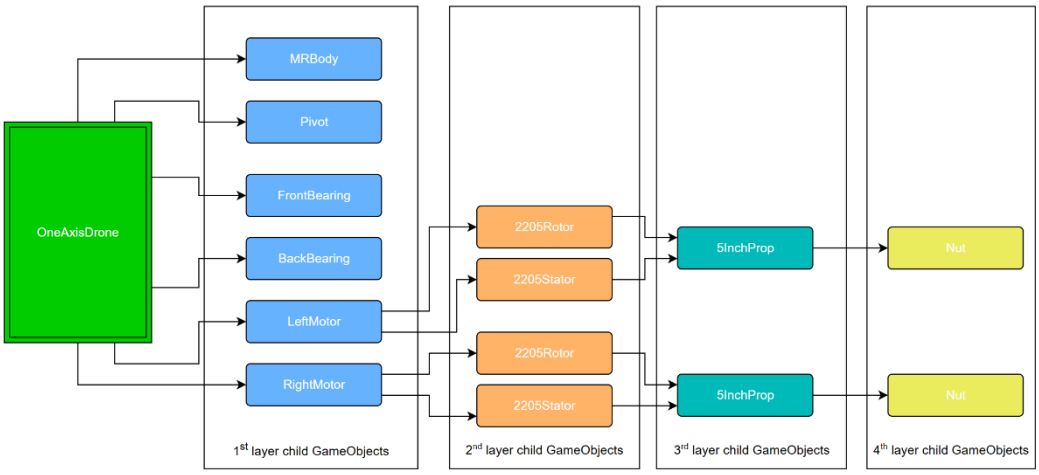


Figure 3. *OneAxisDrone GameObject* hierarchy, with colors representing hierarchical depth.

3.3.3. Physics Simulation

The physics simulation of the digital model is governed by the physics engine present in Unity. The parent *GameObject* (*OneAxisDrone*) has a *Rigidbody* component, enabling it to be simulated in the physics engine and to correctly respond to forces applied to it (gravity, disturbances, and lift). In order to ensure that the simulation reflected the expected physical structure, the *Rigidbody's* center of mass was adjusted to where the center of mass should be on a drone (higher than the center of *MRBody*) as motors and props are situated on top of the body.

Since the simulation should allow a single degree of freedom, namely on the roll axis, a configurable joint was used, constrained in all degrees of freedom except rotation around the roll axis. It is not attached to any other *GameObject*, thus fixing the drone to the world.

Control inputs were applied at the position and taking into account the orientation of the two motors, thus generating torque that imparts rotation on the roll axis to the drone. The magnitude of these forces is determined by a script called *OneAxisDroneByMotorForce*, which reads the output of the PID controller and calculates the lift forces required to maintain the drone at the correct roll angle.

Finally, external disturbances were implemented in the form of turbulence balls. They are lightweight spheres (50 g by default) that are spawned over the drone and fall, colliding with the drone, thus imparting a visually intuitive perturbation to its roll angle. This feature is vital for the serious game as it allows students to evaluate the robustness of their tuning, thus completing the core loop of the serious game.

3.3.4. One-Axis Drone Digital Model Implementation

The digital twin-inspired model is governed by a set of custom C# scripts attached to the parent *GameObject* and its motor child *GameObjects*.

Figure 4 shows the *GameObjects* to which all major scripts that make up and control the digital model are attached. In their totality, they simulate the control system, where the PID controller calculates the desired action and the *OneAxisDroneByMotorForce* reads it and actuates the two motors. In this way, the scripts regulate the drone's roll angle and maintain it to the desired value.

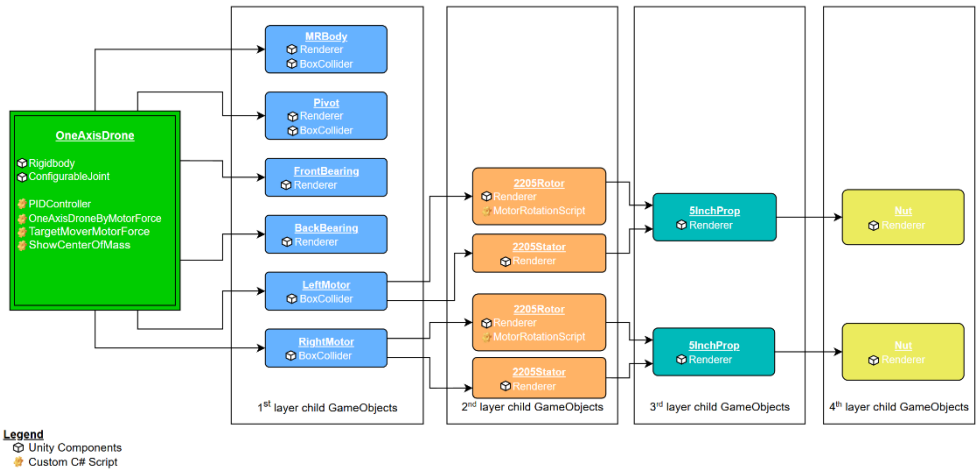


Figure 4. Major custom C# scripts and *GameObjects* holding them, with colors representing hierarchical depth.

1. PID Controller: The PID-controller logic is situated in the script called *PIDController*. The PID controller, in its continuous form, is expressed as:

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau)d\tau + K_D \frac{de(t)}{dt} \tag{1}$$

where *e(t)* is the error between the target and the current angle, and K_P, K_I, and K_D are the proportional, integral, and derivative gains. The error is calculated by subtracting the current value from the target value. However, since the simulation runs in discrete time, the equation looks like this:

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau)d\tau + K_D \frac{de(t)}{dt} \tag{2}$$

To ensure stability in cases when the error persists for long periods of time, the implementation includes an anti-windup mechanism, which clamps the integral term between a minimum and maximum value. Furthermore, the final output of the PID controller is clamped between configurable limits in order to prevent unrealistic values. Normally, for the derivative term, the change in error is calculated and used. However, when the target angle suffers a sudden change, this implementation can cause the derivative term to reach very high values for one frame, resulting in a derivative kick. To mitigate this effect, the derivative term was instead computed from the measured rate of change in the roll angle. By calculating the rate of change from the process variable, we ensure that the derivative term responds only to changes in the system state, and not to jumps in the target value.

2. Motor force application and visual feedback: The script called *OneAxisDroneByMotorForce* has the purpose of taking the output of the *PIDController* script and actuating the motors. At each physics step (*FixedUpdate* in Unity), it computes the required thrust for each motor by scaling the normalized controller output with a maximum thrust constant and clamping the values to ones accurate to a generic 2205 brushless motor. A minimum thrust is maintained as normal brushless motors of drones always keep a minimum RPM. The computed thrusts are applied at the motor position, taking into account the motor orientation, based on the current drone orientation. This application of thrust at motor positions generates the required torque for controlling the rotation on the roll axis while the *ConfigurableJoint* component constrains all degrees

of freedom except for the roll axis.

Since this script already reads the output of the *PIDController* and computes the forces applied to motors, it made sense to extend it to also compute the motors' rotational speed. The maximum speed is determined from parameters of real-life motors (KV value) and the battery powering them (battery cell count), while the instantaneous speed is set proportionally to the determined thrust. The speed values are passed to the *MotorRotationScript*, ensuring consistency between the visual representation and simulated physics. Even though the motors and propellers are not required for the drone to function correctly, they are integral to strengthening the student's perception of motor activity and its effect on the whole system, making the serious game more pedagogically effective.

3. Target angle control: *TargetMoverMotorForce* is the script that sets the value that the PID controller uses as the target. It allows the developer (and the player, through the UI) to set it manually to a fixed value or to set rules for it to oscillate between two values, with the desired speed. The script increases the flexibility of the serious game by enabling demonstrations of both static and dynamic targets.
4. Simulation flow: The scripts described above make up the digital model that sits at the forefront of the serious game, linking user control logic and physics-based response with user interaction. Each script is responsible for a specific function, such as computing control actions, applying forces, animating visual elements, or introducing disturbances, and together they form and control a coherent digital model that allows students to experience how parameter tuning affects system behavior.

Figure 5 shows the flow of information and actions between elements of the simulation. The player defines a static or dynamic target (or leaves it at the default value of zero), which is then compared by the drone's current roll angle, obtained from the *Rigidbody* component. Their difference constitutes the error, which is further used by the PID controller to compute the control output. The resulting value is read by *OneAxisDroneByMotorForce*, which translates it into differential thrust values for the two motors. Forces with these magnitudes are applied as if they originated from the motors themselves, generating the torque that rotates the drone on the roll axis. During the next frame, the cycle is repeated.

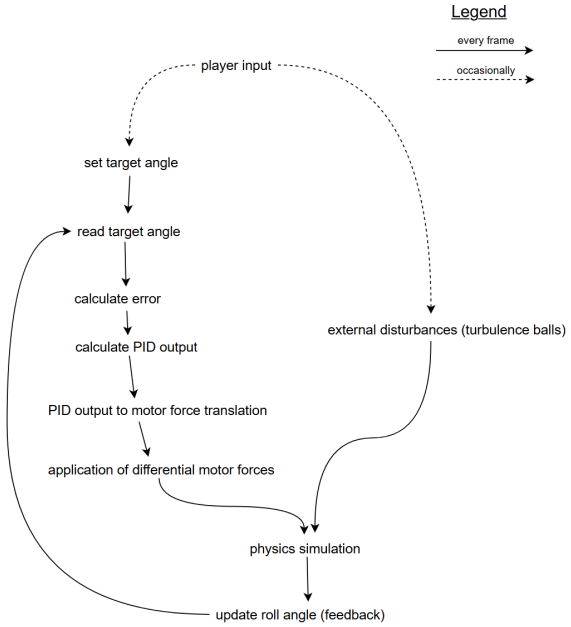


Figure 5. Interaction between components of the control loop in one frame.

External disturbances interact with the system through the physics simulation, and they influence the *Rigidbody's* state, thus eliciting a response from the PID controller, which will try to maintain their desired roll angle. This ensures players receive immediate, visually observable feedback to their tuning efforts.

5. Adjustable parameters: As described in this chapter, a modular approach was used when building the drone simulation, with each component containing several parameters that can be modified, some by the developer, inside Unity, and other by the programmer, inside the UI. Table 1 summarizes the main adjustable parameters and their functions, along with whether they are available only for developers or also for players.

Table 1. Adjustable parameters of the one-axis drone digital model.

Parameter	Description	Editable by
Proportional gain	Elicits a response that is proportional to the error	Player (UI)
Integral gain	Reacts to steady-state error	Player (UI)
Derivative gain	Dampens output based on error change	Player (UI)
Integral saturation	Clamps the integral term to prevent windup	Player (UI)
Target angle	Desired roll angle of the drone if oscillation is disabled	Player (UI)
Target movement speed	Rate at which the target value oscillates if automatic mode is enabled	Player (UI)
Turbulence ball mass	Controls disturbance strength during collisions	Player (UI)
Controller output limits	Clamps the PID-controller output	Developer
Motor KV rating	For determining the max RPM of the motor (visual only)	Developer
Battery cell count	For determining the max RPM of the motor (visual only)	Developer
Maximum motor thrust	Defines maximum force a single motor can produce	Developer
Minimum thrust percentage	Ensures propellers never stop fully, maintaining stability and visual realism	Developer

This split in configuration responsibility allows developers to tailor the simulation to reflect more than one physical twin configuration (different motors, batteries, props, and controllers), while at the same time enabling players to learn by experimenting with different tunes and disturbance scenarios in a safe and accessible manner.

3.3.5. External Disturbance Simulation

In order to fully benefit from a digital twin-inspired model that can be customized by developers and tuned by players, the game offers the option to test the tuning against external disturbances. This functionality is implemented through the *BallSpawner*, which spawns spherical objects called turbulence balls above the drone. Upon spawning, the ball falls and collides with the drone, thus imparting a force upon it, which allows the player to visually observe the behavior of their tuned drone. The mass of the ball can be configured in the user interface so players can see how lighter or heavier disturbances affect system stability and response.

For pedagogical reasons, the choice was made to use falling balls as disturbances. This is because collisions are highly visible, and its effects can help players interpret the effectiveness of their PID tuning. A potential future improvement to this style of applying disturbance to the drone would be to also scale the balls visually, according to their assigned mass, thereby strengthening the connection between the visual cue and the physical property it represents.

In Figure 6, we can observe turbulence balls falling (a) and colliding (b) with the drone. This introduces external disturbances that can be clearly and intuitively observed by vision alone.

Although the current disturbance implementation provides a clear and visually expressive method for visualizing disturbances, it does not resemble any real-world challenge faced by drones, the most common being gusts of wind. Adding player-controlled gusts of wind would improve realism, but it introduces the challenge of making wind, which is an inherently invisible phenomenon, visually observable. Additional visual cues such as dust particles, foliage movement (if outdoors), or subtle, cartoon-like visual effects usually associated with air movement would need to be implemented to convey wind presence and power to the player. The current choice of turbulence balls represents a practical compromise between pedagogical effectiveness, implementation simplicity, and realism.

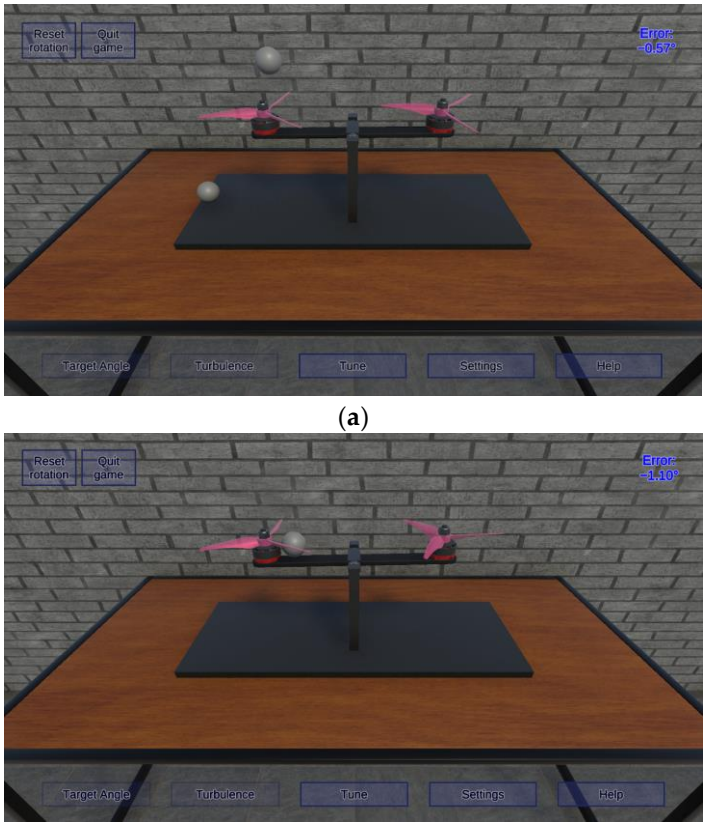


Figure 6. Turbulence balls falling and interacting with the digital model—(**a**) while ball is falling and (**b**) while the ball is interacting with drone through the physics engine.

3.3.6. User Interface

The user interface provides easy access to the simulation's main functions by offering rapid access to all the essential functions and parameters needed for experimenting with the digital model.

Figure 7 illustrates the in-game interface, which is controlled by the bottom panel containing five buttons, each opening its respective panel.

The panels seen in Figure 7 and explained in Table 2 offer comprehensive control over the simulation, allowing players to adjust the PID-controller values, to set the desired target (static or dynamic), and to also introduce external disturbances. This compact and intuitive interface ensures that students have all the tools needed to learn through experimentation at their disposal.

Auxiliary buttons are included for quitting the game and for resetting the drone rotation. The latter is especially useful when the student inputs values into the PID controller that would cause uncontrollable movement that breaks the simulation. One way that the simulation can break is by getting the drone rotating so much that it becomes stuck

in the table. While this rarely happens, an easily accessible button fixes this and any other issue that may potentially arise.

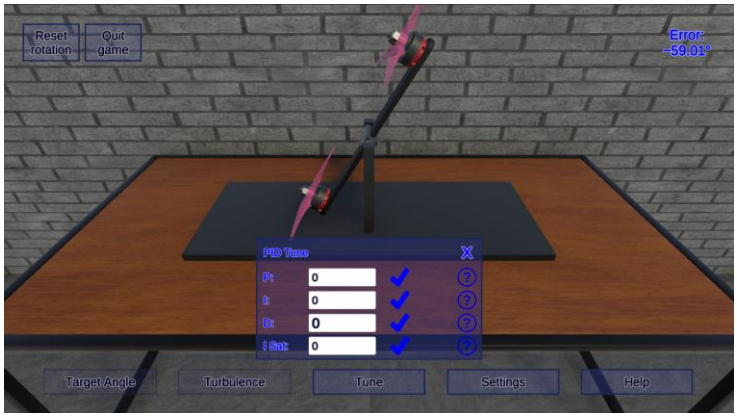


Figure 7. In-game interface, with the Tune panel open.

Table 2. Panels allowing the player to interact with the digital model.

Description	Panels
Allows players to set a static target value or generate automatic oscillations	Target angle panel
Allows players to set the mass of the turbulence balls	Turbulence panel
Allows players to modify K_P, K_I, K_D, and ISat (integral saturation)	Tune panel
Will contain settings in future releases.	Settings panel
Provides short instructions describing the main actions undertaken by the player	Help panel

4. Results

4.1. Testing Methodology

4.1.1. Goal and Research Questions

The goal of the study was to check if a serious game containing the digital twin-like model of a hypothetical one-axis drone can support instructors during PID teaching. Specifically, the serious game's usefulness in diagnosing students' tuning difficulties was evaluated using metrics collected in-game as the game was designed to provide a safe, low-risk environment for students to tinker in. On top of this, a confirmatory analysis was performed to check if unsupervised sessions can also produce measurable gains in tuning performance.

To better meet the stated goal, the following research questions were addressed:

- RQ1: Do error values provide actionable diagnostic signals that can help instructors identify PID tuning issues that students are having?
- RQ2: Does a short, unsupervised tinkering session improve quiz scores and stability-oriented metrics?
- RQ3: Does tinkering effort decrease from the first tinkering session to the second one without degrading stability-oriented performance?

Research question 1 will be answered in an exploratory manner. For this, metrics will be plotted and conclusions will be drawn. When discussing the error values, we will show how common PID issues manifest themselves. In the confirmatory part (research questions 2 and 3), pre/post quiz scores and stability-oriented metrics will be formally tested across the two tinkering sessions.

4.1.2. Participants and Context

There were a total of 21 participants in this study, aged 18–33, all part of a lab session at an international student festival in Timisoara (ISWinT 2025). The data was collected using anonymous IDs. The students had very diverse cultural and technical backgrounds, along with different prior PID-controller knowledge. The backgrounds are as follows: computer science, multimedia, aerospace, and electrical engineering. The field with the most students was computer science, with 14 students. Multimedia and electrical engineering had 3 students each, while aerospace contained a single student.

4.1.3. Study Design

The study followed a within-subjects design, with each student serving as their own control across the quizzes and two rounds of playing the serious game under identical task conditions. As allowing students to change the target angle would make identifying the error from analytics data impossible without additional information, the target angle was set to 0 degrees. The option to customize the turbulence balls, while implemented, was disabled in the official testing rounds as the balls came already set to a relevant value, and further changing the mass could have led to incredibly strong or weak disturbances, which would lower their relevance. To reduce the complexity and to maintain focus on fundamental concepts, participants were advised not to modify the integral gain and saturation as they were not treated in-depth in the theory part due to time constraints.

The participants were instructed to tune the PID controller until the system stabilized itself at 0.4° roll angle. Optionally, they were informed of the ability to spawn turbulence balls to observe the tuned system's response and gauge its stability. Tinkering sessions were limited to 10 min, after which the game would automatically stop, while also allowing any student who was satisfied with their result to quit early via a dedicated button. Upon quitting (automatically or as a choice), the game sent the collected analytics data to the back-end.

4.1.4. Tools

Data was collected using two different means: forms and metrics telemetry, giving us both subjective, student questionnaires, along with objective metrics that could be analyzed.

For making quizzes, Google Forms have been used for their ease of setup, distribution, and because they offer the possibility to export to CSV files. Students were given the option to choose between randomly generated IDs, which were present in all quiz exports and both analytics results, enabling the merger of all data collected from a user. IDs are vital as the study relies on measuring differences between different stages of knowledge. There is no way of associating the real student to a username available as the choice was neither tracked not written anywhere, and the data provided is not enough to make any link between a real person and a username.

A custom back-end was developed for logging analytics data to ensure full control over recorded data. Data is saved locally, on the author's private server, with no reliance on third-party services, and can be easily exported to JSON files. The back-end also relied on the IDs for merging data for the same user, so no personally identifiable information was recorded. The implementation details of the custom back-end are outside the scope of this paper. Later subsections will describe the relevant information, mainly the data captured and its usage.

4.1.5. Procedure

The study consisted of multiple steps, provided in fixed order and undertaken in two days, with the first five steps being performed on the first day, and the last two on the second day.

Step 1: Background quiz

Students were given a short survey for understanding their background and matching it to a unique ID. The survey contained questions about their technical background, country of origin, country they were currently studying in, their age, and their prior experience with PID controllers.

Step 2: 30-min theory/practical course

A concise course covering PID controllers was taught. It presented topics ranging from the need for having these controllers to the math needed to understand why they work the way they do. This was followed by two examples of PID controllers unrelated to the app and was used to build intuition about PID controllers and the way they work.

Step 3: App introduction and controller functioning quiz

The serious game was presented, showing the available features and highlighting the ability to change the parameters for tuning the digital model. Following the presentation, students completed a quiz targeting their understanding of parameters related to and controlled by the PID controller (controller's goal, error computation, and how the PID output is used). These questions align directly with the game's learning objectives.

Step 4: First tinkering phase

Students were instructed to tune K_P, K_I, and K_D to stabilize the roll angle as close to 0° as possible. While they were working, the custom back-end recoded the following session parameters:

- error value in degrees for the entire session
- last error value of the sessions
- total session duration in seconds, up to a maximum of 600 s, as the game stopped at that time
- number of parameter changes performed during the session

Step 5: Post-tinkering quiz

Using the same questions as the previous quiz, this quiz aimed to gauge the impact that tinkering with a digital reproduction of a real system had on the understanding of core concepts.

Step 6: Second tinkering phase

Students replayed the serious game and tuned it again, under the same conditions as the previous tinkering phase. The second set of data was to be compared to the first one in order to observe if playing the serious game improved the students' ability to tune a PID controller.

Step 7: Final quiz

At the end of the study, students were asked to self-assess the improvement in their understanding of the inner workings of the PID controller, and to provide feedback in the form of improvements to the app that could increase teaching effectiveness.

4.1.6. Collected and Derived Metrics

The multiple datasets were merged using the anonymous unique IDs. Quizzes provided participants' background details (age, country of origin, country of study, technical

background, and prior PID exposure) and pre- and post-tinkering-session self-assessed scores for their understanding of the PID controller, along with feedback on the serious game. Figure 8 shows the format of the JSON file in which all quiz-provided information was saved. The tinkering sessions contributed with metrics taken from inside the game, which can be grouped into two sections: collected (values of game state, sent directly to the analytics back-end and derived) and determined through various means from the collected metrics.


```
{
    "username": "username",
    "Quiz1":{
      "What is your background? (programming, automation, media, art etc.)":"background",
      "Where are you from (country)?":"country",
      "Where do you study (country)?":"country",
      "How old are you?":0,
      "Do you have any experience with PID controllers?":"Yes/No",
      "By this, I give my consent for the data collected with the fictitious user to be collected and processed.  ":"Yes/No"
      },
    "Quiz2":{
      "What is the goal of the PID controller in the example app?":"Answer",
      "Which parameter is the PID controller trying to regulate?":"Answer",
      "How is the error calculated in the PID controller?":"Answer",
      "How is the PID controller output used?":"Answer"
    },
    "Quiz3":{
      "What is the goal of the PID controller in the example app?":"Answer",
      "Which parameter is the PID controller trying to regulate?":"Answer",
      "How is the error calculated in the PID controller?":"Answer",
      "How is the PID controller output used?":"Answer"
    },
    "Quiz4":{
      "Do you have a better understanding of the goal, workings, and usage of a PID controller?":"Yes/No",
      "Has your understanding increased by using the digital twin or was it enough after the first step?":"Yes/No",
      "What would you improve in the app to increase its usefulness for teaching PID controllers?":"Answer"
    },
    "collectionDate_Before":"yyyy-mm-dd-tt",
    "data_Before":{
    "ErrorValues_Before":[0,0,0],
    "LastSecondError_Before": 0,
    "PlayTimeSeconds_Before": 0,
    "ParameterChanges_Before": 0
    },
    "collectionDate_After":"yyyy-mm-dd-tt",
    "data_After":{
    "ErrorValues_After":[0,0,0],
    "LastSecondError_After": 0,
    "PlayTimeSeconds_After": 0,
    "ParameterChanges_After": 0
    }
}
```


Figure 8. JSON file structure.

Quizzes recorded the following data:

- Quiz 1 (background information): age, country of origin, country of study, technical background, and prior exposure to PID controllers.
- Quiz 2 (knowledge application on the digital model's PID controller): purpose of the PID controller, error computation, and how the PID controller is used by the system.
- Quiz 3 (post tinkering knowledge application on the digital model's PID controller): same format as Quiz 2, enabling direct pre–post comparison.
- Quiz 4 (final quiz): self-assessment of the perceived improvement in understanding PID controllers, plus open-ended feedback on improving the serious game.

The custom analytics back-end recorded the following metrics:

- Submission time: sent at the end of the tinkering session. Contains the date and time of the end of the current tinkering session.
- Play time: sent at the end of the tinkering session. Contains the time that the player spent in-game, in seconds.
- Number of parameter changes: tracked throughout the tinkering session and sent at the end. Contains the number of times the player applied a change to the values used by the PID controller.

- Error values: tracked throughout the tinkering session and sent at the end. Contains the value of the roll angle, sampled at 10 Hz for the entire tinkering session. Stored as a list of values in degrees.
- Final error: sent at the end of the tinkering session. Contains the last recorded error value in degrees.

From the collected metrics, the following derived metrics were computed:

- Stability threshold: the tinkering phase instructions stated that an error of less than 0.4° was set as the goal to be reached by the students and was considered good enough for the purpose of this study. For this, the minimum and maximum values present in the earliest 10 s section with the lowest error value were calculated, along with the mean error value on that segment and its starting point in time.
- Normalized area under absolute error: this metric was devised to provide a session-wide stability summary for easy comparison between sessions. It was calculated by taking the absolute value of all collected error values, followed by calculating the area under them. Finally, the obtained value was time-normalized (per minute) as comparisons between sessions must not be affected by session length.
- Parameter edit rate: because participants could end the tinkering session earlier than the maximum allocated time of 10 min, the parameter edit rate normalizes their effort by time spent in-game. It is calculated by dividing the number of parameter changes perform during a tinkering stage by the total play time.

While disturbance events (spawning of turbulence balls) were not logged explicitly, their influence is visually apparent in error plots as short-lived increases in error values, followed by quick re-stabilization. Re-stabilization time also tells us if a system has reached stability by waiting until oscillations were small enough, or if the system was tuned correctly, without oscillations. Since spawning turbulence balls was optional, this is something we can observe in only some of the metrics data. A drawback to the lack of disturbance event logging is that they can be observed through the error values only on a system that is at least partially stable.

Per-participant differences between the two tinkering sessions were calculated and analyzed in the data interpretation subsections to summarize within-subject change under identical conditions.

4.2. Data Interpretation

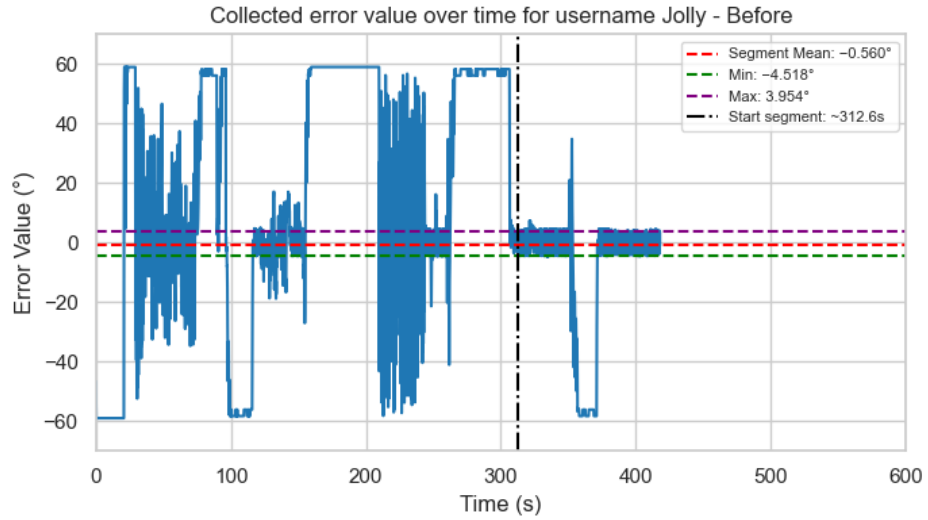
This section contains the analysis of the collected data with respect to the previously stated research questions. The diagnostic value of collected analytics metrics (RQ1), the effects of unsupervised tinkering on knowledge (RQ2), and improvements in efficiency without degradation of tuning performance (RQ3) will be explored.

4.2.1. RQ1—Do Error Values Provide Actionable Diagnostic Signals That Can Help Instructors Identify PID Tuning Issues That Students Are Having?

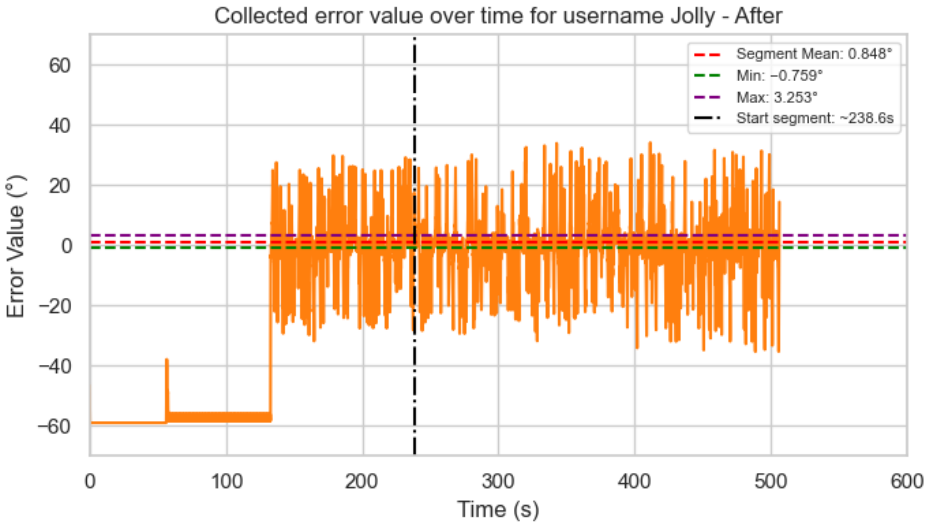
The collected error values sampled at 10 Hz can be easily plotted and then visually analyzed alongside each other for both the day 1 and day 2 sessions (before and after). This was performed for each student to see if practical hints can be prepared by an instructor to help students overcome potential PID tuning issues. In the following paragraphs we will explore the error values of three students and see what conclusions about their knowledge can be drawn. Each plot shows the initial position of the drone. When the game starts, the drone is laying on its side, with a roll value of around $-60°$.

Figure 9a shows the error values recorded from a computer science student during the first tinkering session (before). In it, we can observe a high error value, which reaches even negative or positive 60°. This clearly indicates a very weak power applied to the digital

model, likely as a result of a K_P value that is too small for the system. This implies a lack of understanding of the main concepts, including the proportional term of a PID controller. This is in contrast to the second tinkering stage (after), presented in Figure 9b, in which the one-axis drone maintains an oscillation of $\pm 30^\circ$. From the plot shape we understand that the system has an adequate K_P value, since it has enough power to overcome gravity, but the K_D term is tuned incorrectly, as shown by the massive overshoot. Both plots contain an additional four terms related to the best (smallest) continuous 100 error values: min error value of the segment, max error value of the segment, mean of the error values present in the segment, and the start time of the segment. This was calculated to more reliably highlight potential differences in the error. In this case, the error value clearly improved, since the distance between the lines determined by the minimum and maximum values of the segment was lowered significantly. After the first session, an instructor can easily identify an inadequate P term and help the student focus on that concept first. After the second session, the instructor should encourage the student to revise the PID-controller theory with a focus on the D term in order to improvement tuning in a potential third session.



(**a**)



(**b**)

Figure 9. Computer science student's (username: Jolly) error value over time. (**a**) Before and (**b**) after.

The analysis of this student's error values shows an acceptable understanding of the PID controller from the start (Figure 10a), with errors between $\pm 20^\circ$ for the most part

during the first session. This shows that the student has easily learned how to tune the P term, but still struggles with tuning the D term. The second session, however, shows a near-perfectly tuned system, with the best segment (defined by the best 100 continuous error values) being between -0.053° and 0.022°. Small, temporary, and negatively inclined deviations from stability can be observed even after the system has been tuned. They are the result of the player choosing to spawn the aforementioned turbulence balls with the purpose of testing the system's stability when disturbed by outside forces. The system handles both short, singular turbulence balls (observed as short-lived deviations) and large groups of balls (observed as longer-lasting deviations) with minimal oscillations and overshoot. Finally, the analysis reveals that the player required significantly less time to tune the system while reaching higher tuning performance.

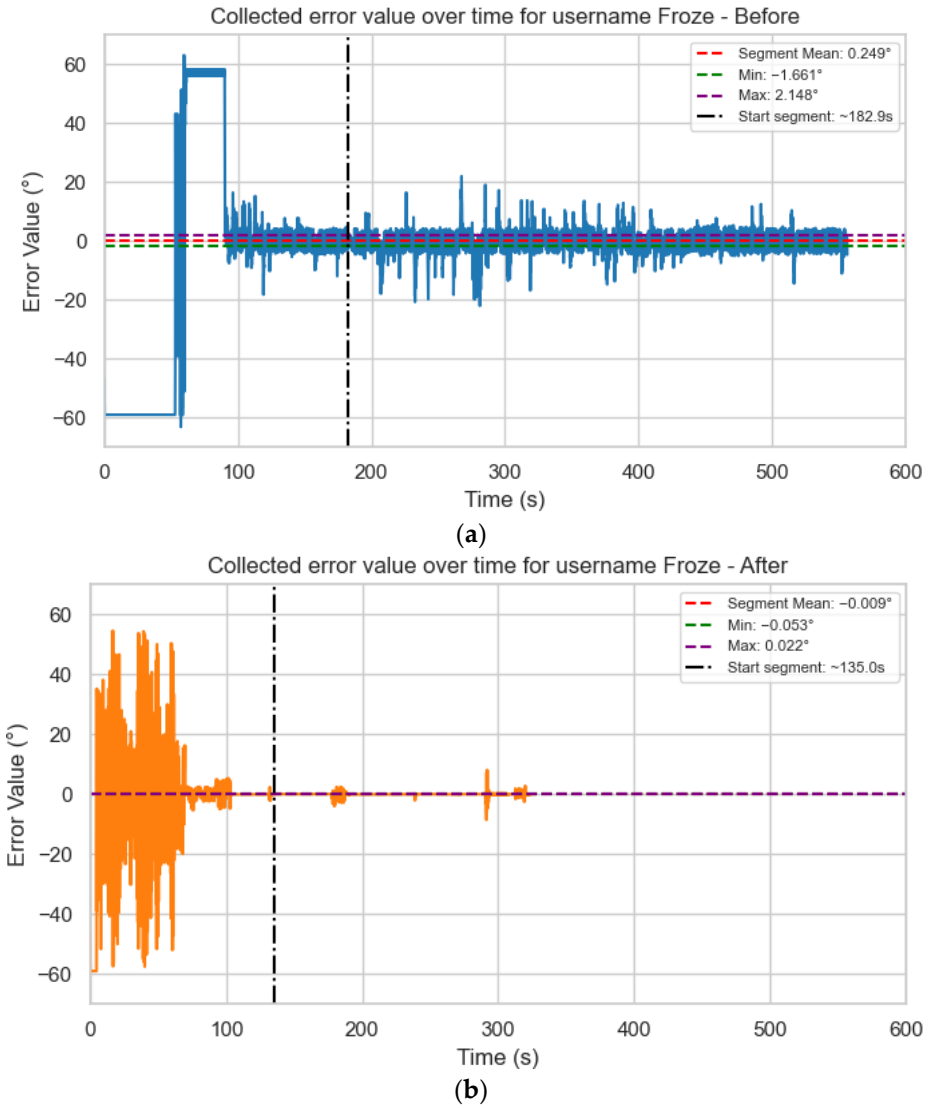
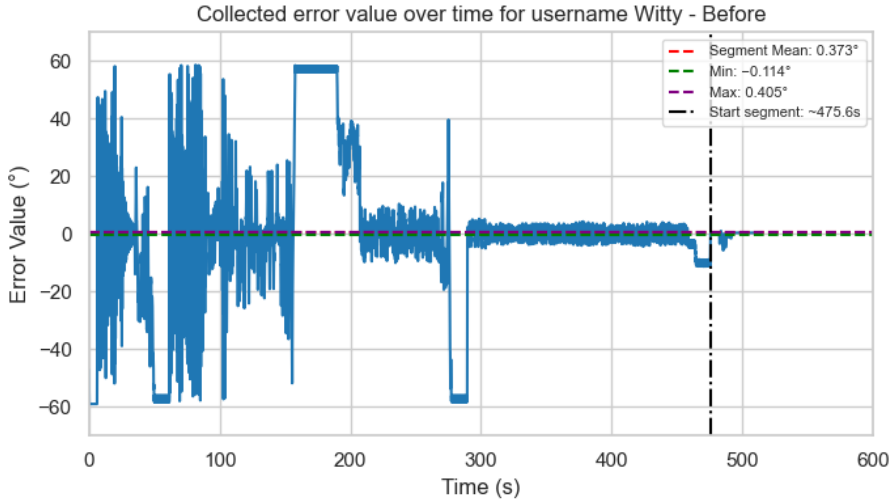


Figure 10. Computer science student's (username: Froze) error value over time. (**a**) Before and (**b**) after.

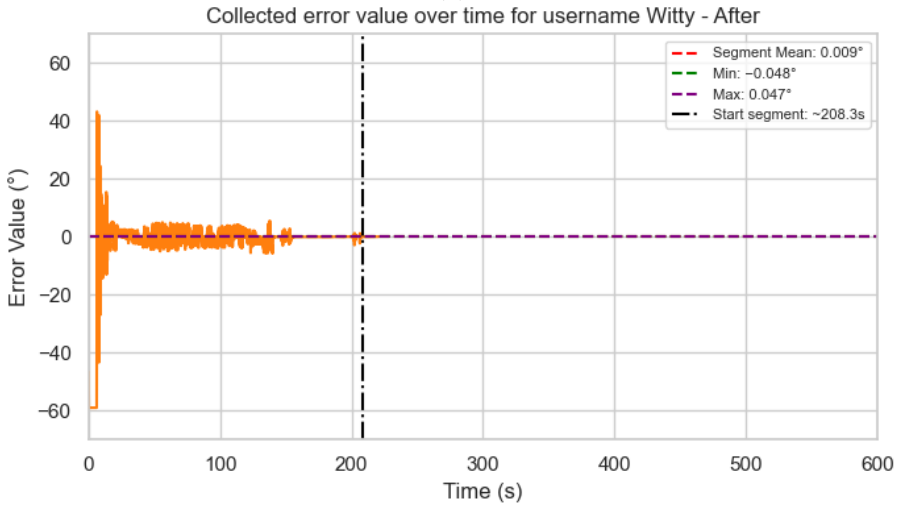
While the previous two students had a computer science background, the student whose error is plotted in Figure 11a has a multimedia background. A struggle to obtain the error to improve can easily be observed in the first half of Figure 11a, followed by a somewhat stable period 300 s after the session's start. An acceptable understanding of PID controllers is, therefore, present even after the first tinkering session, with instructors well placed to advise the student to improve K_D. It can be observed that the knowledge

is retained until the second tinkering session as the student quickly brings the error in the same interval as in the first session. Then, after about 150 s, the student starts tuning the D term, thus lowering the error to near-perfect values. After this stage, the controller is tuned correctly and responds to external disturbances well.

These example analyses show how information can be inferred from the plotted error values. Instructors can identify patterns such as drift, overshoot, and jitter and point students to their likely cause and give targeted advice after each attempt. In many cases, the second session either reduces the error or reaches the same error in significantly less time. Overall, the serious game works best as a supervised learning tool that helps focus feedback rather than as a standalone tutor. For information on all students who participated in this study, the full set of plots can be consulted in the provided Git repository [47].



(**a**)



(**b**)

Figure 11. Multimedia student's (username: Witty) error value over time. (**a**) Before and (**b**) after.

4.2.2. RQ2—Does a Short, Unsupervised Tinkering Session Improve Quiz Scores and Stability-Oriented Metrics?

In order to see if unsupervised sessions of playing the serious game helped students improve the theoretical understanding of PID controllers, a score was calculated for each student who provided answers to Quiz 2 (administered before playing the serious game) and Quiz 3 (administered after playing the serious game for the first time). Responses were

assigned scores that reflected the percentage of correct answers, which means that the only possible values were 0, 0.25, 0.5, 0.75, and 1.

Students scored 0.666 on average before the first tinkering session, and 0.619 after the first tinkering session. This leads to an insignificant-looking mean change of −0.047. To check the true relevance of this change, the Wilcoxon signed-rank test was chosen as an inferential procedure since it is suitable even when data is discrete and does not follow normal distribution [48].

Table 3 was computed while performing the Wilcoxon signed-rank test. The null hypothesis was H0: there is no systematic change in scores. The alternative hypothesis was H1: there is systematic change in scores. Since statistical analysis resulted in W = 53.5, with $p = 0.479$, we failed to reject the null hypothesis. Therefore, we can conclude that there is no systematic change in scores.

Table 3. Values obtained while performing the Wilcoxon signed-rank test on Quiz 2 and 3 scores.

Total Score Quiz 2	Total Score Quiz 3	Difference	Absolute Difference	Difference Sign	Rank	T_-	T_+	W
0.75	0.75	0	0	0	-			
1	1	0	0	0	-			
0.75	0.75	0	0	0	-			
0.75	0.75	0	0	0	-			
0.75	0.75	0	0	0	-			
0.75	0.5	−0.25	0.25	−1	6.5			
0.75	1	0.25	0.25	1	6.5			
0.75	0.5	−0.25	0.25	−1	6.5			
0.75	1	0.25	0.25	1	6.5			
0.25	0.5	0.25	0.25	1	6.5	82.5	53.5	53.5
0.75	0.5	−0.25	0.25	−1	6.5			
0.75	1	0.25	0.25	1	6.5			
1	0.75	−0.25	0.25	−1	6.5			
0.25	0.5	0.25	0.25	1	6.5			
0.5	0.25	−0.25	0.25	−1	6.5			
0.25	0	−0.25	0.25	−1	6.5			
0.25	0.5	0.25	0.25	1	6.5			
0.75	0.25	−0.5	0.5	−1	14.5			
0.75	0.25	−0.5	0.5	−1	14.5			
1	0.5	−0.5	0.5	−1	14.5			
0.5	1	0.5	0.5	1	14.5			

The effect size, expressed as a rank-biserial correlation was −0.213. This indicates a negligible difference, while the median change was 0, showing that, in fact, most students did not change their score between the two quizzes.

For checking improvements in the measured metrics, the error values were used to calculate the normalized area under absolute error. This derived metric is calculated by taking the absolute value of all collected error values, calculating the area under them, then normalizing it to be comparable regardless of time spent in-game. These metric aggregates measure deviation from 0° across the whole attempt and are comparable even when students quit the tinkering session early. Using this metric, a Wilcoxon signed-rank test was performed across the two tinkering sessions as it is appropriate for discrete data that does not follow a normal distribution.

Table 4 shows the values resulting from computing the Wilcoxon signed-rank test. The null hypothesis was H0: there is no systematic change in the normalized area under absolute error, while the alternative hypothesis was H1: there was systematic improvement.

A two-tailed Wilcoxon test reported $p = 0.3737$ and W = 89. With a critical rank sum of 67, we fail to reject H0. This means that unsupervised use does not lead to a detectable reduction in normalized area under absolute error between sessions.

Taken together, the confirmatory tests show no measurable gain from short, unsupervised tinkering sessions as both quiz scores and error values did not suffer improvements between sessions. Consequently, this serious game should be viewed as an auxiliary learning tool to be used with instructor guidance, where error values are interpreted by the instructor and transformed into actionable feedback for the students.

Table 4. Values obtained while performing the Wilcoxon signed-rank test on the area under error value.

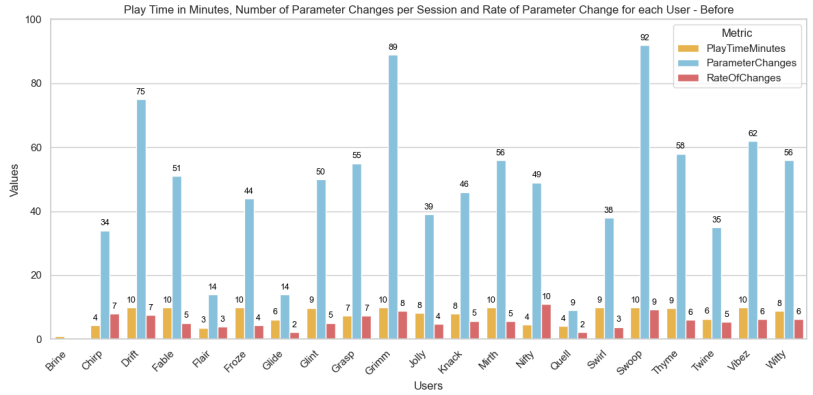
Day1_nAUE	Day2_nAUE	Difference	Absolute Difference	Difference Sign	Rank	T_-	T_+	W
243.30624	228.72069	−14.585555	14.585555	−1	1			
864.69223	901.28067	36.588443	36.588443	1	2			
894.53042	938.78995	44.25953	44.25953	1	3			
201.01026	326.56464	125.55438	125.55438	1	4			
979.56527	848.03606	−131.529215	131.52922	−1	5			
582.98658	776.97678	193.9902	193.9902	1	6			
676.55098	923.2512	246.70023	246.70023	1	7			
1063.6568	783.04613	−280.610644	280.61064	−1	8			
1025.5287	741.60723	−283.921514	283.92151	−1	9			
1189.2225	1514.816	325.59347	325.59347	1	10	142	89	89
1159.6068	776.09	−383.51679	383.51679	−1	11			
695.76433	289.4231	−406.341222	406.34122	−1	12			
962.47359	512.89044	−449.583156	449.58316	−1	13			
970.08941	497.38339	−472.706021	472.70602	−1	14			
1768.0974	1272.3261	−495.77132	495.77132	−1	15			
833.1392	196.43131	−636.707888	636.70789	−1	16			
1448.2651	666.45058	−781.81449	781.81449	−1	17			
694.37792	1500.5228	806.14484	806.14484	1	18			
796.06865	1620.2031	824.13441	824.13441	1	19			
1611.9923	2501.8061	889.81381	889.81381	1	20			
3491.1501	137.16073	−3353.989352	3353.9894	−1	21			

4.2.3. RQ3—Does Tinkering Effort Decrease from the First Tinkering Session to the Second One Without Degrading Stability-Oriented Performance?

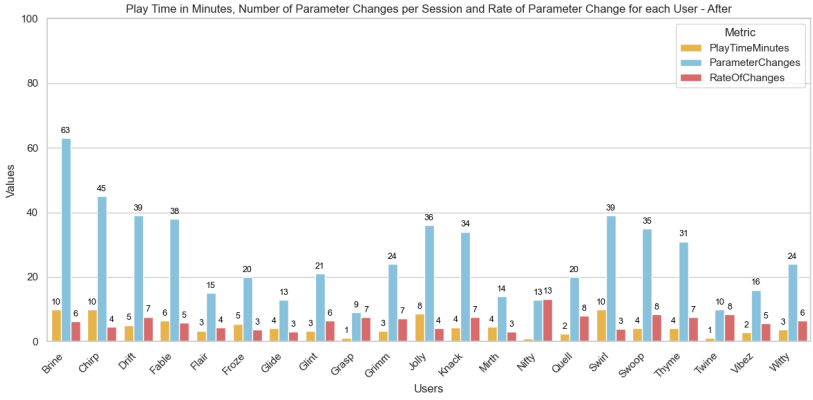
To evaluate students' efficiency in tuning the PID controller, per-participant effort between the sessions was compared. The effort was measured directly by two metrics: the number of parameter changes per session and play time. The rate of change was derived from the previously mentioned metrics and shows the effort normalized by play time.

As shown in Figure 12, almost all students spent significantly less time during the second session. Many also performed fewer parameter changes, while keeping a somewhat stable rate of change. This implies that students worked at a similar per-minute pace but finished sooner and by performing fewer edits. This is consistent with the quicker error convergence observed visually during the analysis of the second session performed while answering RQ1.

As reported in the answer to the second research question, while there was no systematic change in quiz scores and no reliable improvement in error, there was also an absence of worsening of performance. This demonstrates that the reduction in effort did not come at the expense of stability, so students simply reached satisfactory PID tunes faster.



(**a**)



(**b**)

Figure 12. Bar charts containing the play time, number of parameter changes, and rate of change. (**a**) Before and (**b**) after.

5. Conclusions

This paper explored a serious game built around the digital twin-inspired model of a hypothetical one-axis drone, developed with the goal of supporting instructors when teaching PID controllers. The primary contribution is the simplified, safe, and free to deploy game, in which students can tinker with K_P, K_I, and K_D, while error values are sent to an analytics back-end for later analysis. The simplified drone with one degree of freedom keeps concepts approachable while reducing cognitive load, which makes tuning the system easier. Using a serious game comes with significant advantages over a physical system, such as eliminating safety risks and avoiding additional hardware and maintenance costs. The digital model of a simplified drone with a single degree of freedom proved effective for teaching PID tuning fundamentals as it enabled approachable and repeatable experiments while avoiding cognitive load in students. Turbulence balls spawned over the drone fulfilled their goal of imparting predictable and visually observable disturbances on the drone.

The research questions are revisited below, and the performed analyses are summarized. Collected data was explored to see what metrics can tell instructors regarding whether brief, unsupervised tinkering sessions yield significant gains and whether efficiency improves without harming stability.

RQ1: the first research question asked whether error values provide actionable diagnostic signals that help instructors identify PID tuning issues that students have. The analysis showed that error values collected at 10 Hz can be interpreted visually, from a plot, to identify error patterns and to generate clear, actionable hints for the students.

RQ2: the second research question asked whether a brief, unsupervised tinkering session shows clear improvements in quiz scores and/or stability metrics. No systematic change was found between pre/post quiz scores and the normalized area under absolute error between the two tinkering sessions.

RQ3: the third research question asked whether tinkering effort decreases from the first to the second tinkering session without degrading stability metrics. Analytics data showed substantially fewer edits and less time spent in the second session, while stability metrics did not suffer any improvement or degradation.

The results proved that the serious game is most efficient when used as an auxiliary tool, under the supervision and guidance of instructors, as they are qualified to quickly diagnose common PID issues and give targeted hints to help consolidate understanding.

The analysis of the data revealed a couple of limitations that affected this study. The number of participants was small, sitting at 21 students, and unsupervised use of the game likely limited learning gains to undetectable levels. The lack of logging of disturbance events and very strict stability thresholds also played a small role in limiting the analysis of the data, while the quiz content provided a low-resolution scale that does not allow measurements of small changes. Further studies will include guided sessions, refined assessments, and qualitative feedback, and will be used to better evaluate learning outcomes.

Nevertheless, the analysis yielded useful insights into how digital twin-inspired models can support and enhance the teaching of automation and control concepts. Lessons learned from this study form a solid basis for future research aimed at the development of scalable teaching tools that better prepare students for the demands of Industry 4.0 and 5.0.

Data from both the quizzes and the analytics were collected anonymously, with participants giving their explicit consent to data collection with the fictitious user ID. All analytics data is available in the public repository as a JSON file. Furthermore, plots comparing analytics metrics for all students (plots and bar charts), two Python scripts (version 3.11.3) for regenerating the plots and bar charts, a CSV file that has all relevant values from the plots, and two Excel files detailing how the two Wilcoxon signed-rank tests were performed are available.

Future work will involve building an instructor dashboard that plots live error values, calculates derived metrics, and highlights students who might require help. Parameter change and disturbance events' timestamps will be logged for facilitating a more accurate study of error values. Tuning performance testing will be separated from the tinkering scene to ensure identical test conditions across students. A wind model with clear visual cues (moving leaves and dust movement effects) is planned to extend turbulence generation and enhance realism. For students, a cheat sheet with common tuning problems, the error shape associated with them, and how they can be fixed, will be provided to be used as a reference. Furthermore, the assessment will be refined, with more quizzes being more sensitive to change and the studies being performed on larger groups. Finally, this paper represents only a part of the ongoing research and development in this area as it has not yet progressed past the initial goal of developing a customizable digital model. The next steps will aim to establish a bidirectional connection between the digital model and a physical system. Achieving this link would transform the current standalone simulation into a true digital twin, fully synchronized with its physical counterpart.

Feedback provided by participants in the last quiz mostly aligned with the improvements presented in the previous paragraph. Students suggested clearer guidance during tuning, the ability to visualize error shape in plots, more realistic disturbances, and for the serious game to contain theoretical and tuning information, instead of it being presented separately, before tinkering. These suggestions will guide the next iterations of this research.

In summary, the digital model-based serious game proved to be a practical, safe, and cheaply scalable way to enhance the teaching of PID fundamentals. Even though unsupervised use yielded no performance improvements on its own, repeated exposure improved tuning efficiency without loss in tune performance. When used in tandem with guidance by instructors, the tool can help students apply the theory in a digital twin-inspired simulation, thus laying the foundation for future classrooms that bridge the gap between theory and practice.

Author Contributions: Conceptualization, R.B.; Methodology, R.B. and S.N.; Software, R.B. and S.N.; Validation, R.B. and S.N.; Formal analysis, R.B. and S.N.; Resources, R.B. and S.N.; Data Curation, R.B. and S.N.; Writing—original draft, R.B. and S.N.; Writing—review & editing, R.B. and S.N.; Visualization, R.B. and S.N.; Supervision, H.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: This work is about the design and technical evaluation of a serious game. It is not a clinical study and does not report results from a clinical trial. The activity was conducted with adult volunteers, who gave their informed consent, using anonymous IDs, and with no personally identifiable information being collected. Under our institution's policies for minimal-risk educational research with anonymous data, ethical review was not required.

Informed Consent Statement: Informed consent was obtained from all subjects involved in the study. Electronic informed consent was collected individually and stored alongside the collected data; documentation is available from the corresponding author upon reasonable request. The public repository contains only data collected anonymously. Written informed consent has been obtained from all participants for the publication of anonymous results and illustrative Figures derived from their data.

Data Availability Statement: The dataset supporting this study is openly available in our repository [47] and includes: (i) the complete set of 21 raw session logs merged and saved in JSON format, (ii) two Excel files with the both Wilcoxon signed-rank tests, and (iii) two Python files for data processing and plot generation. All generated plots are also available in the open repository.

Acknowledgments: The authors express their gratitude to the Polytechnic University of Timisoara (UPT) and the student organization Liga AC for their support in carrying out this research. The ISWinT student festival provided a good place to test the serious game with students of varied backgrounds, which contributed significantly to the relevance and quality of the results. Our gratitude also extends to the 21 students who participated in the study.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

VR	Virtual Reality
AR	Augmented Reality
PID	Proportional-Integral-Derivative
DT	Digital Twin
MPC	Model Predictive Control
UI	User Interface
ISWinT	International Student Week in Timisoara
JSON	JavaScript Object Notation
ID	Identifier
CSV	Comma-separated values

References

1. Raave, D.K.; Saks, K.; Pedaste, M.; Roldan Roa, E. How and Why Teachers Use Technology: Distinct Integration Practices in K-12 Education. *Educ. Sci.* **2024**, *14*, 1301. [CrossRef]
2. Kearney, S.; Maakrun, J. Let's Get Engaged: The Nexus between Digital Technologies, Engagement and Learning. *Educ. Sci.* **2020**, *10*, 357. [CrossRef]
3. Lampropoulos, G.; Fernández-Arias, P.; de Bosque, A.; Vergara, D. Virtual Reality in Engineering Education: A Scoping Review. *Educ. Sci.* **2025**, *15*, 1027. [CrossRef]
4. Jaramillo-Mediavilla, L.; Basantes-Andrade, A.; Cabezas-González, M.; Casillas-Martín, S. Impact of Gamification on Motivation and Academic Performance: A Systematic Review. *Educ. Sci.* **2024**, *14*, 639. [CrossRef]
5. Ramírez de Dampierre, M.; Gaya-López, M.C.; Lara-Bercial, P.J. Evaluation of the Implementation of Project-Based-Learning in Engineering Programs: A Review of the Literature. *Educ. Sci.* **2024**, *14*, 1107. [CrossRef]
6. Suleman, F.; Videira, P.; Araújo, E. Higher Education and Employability Skills: Barriers and Facilitators of Employer Engagement at Local Level. *Educ. Sci.* **2021**, *11*, 51. [CrossRef]
7. Christiansen, L.; Hvidsten, T.E.; Kristensen, J.H.; Gebhardt, J.; Mahmood, K.; Otto, T.; Lassen, A.H.; Brunoe, T.D.; Schou, C.; Laursen, E.S. A Framework for Developing Educational Industry 4.0 Activities and Study Materials. *Educ. Sci.* **2022**, *12*, 659. [CrossRef]
8. Reis, V.; Santos Baptista, J.; Duarte, J. Immersive Tools in Engineering Education—A Systematic Review. *Appl. Sci.* **2025**, *15*, 6339. [CrossRef]
9. Tobarra, L.; Utrilla, A.; Robles-Gómez, A.; Pastor-Vargas, R.; Hernández, R. A Cloud Game-Based Educative Platform Architecture: The CyberScratch Project. *Appl. Sci.* **2021**, *11*, 807. [CrossRef]
10. Di Nardo, V.; Fino, R.; Fiore, M.; Mignogna, G.; Mongiello, M.; Simeone, G. Usage of Gamification Techniques in Software Engineering Education and Training: A Systematic Review. *Computers* **2024**, *13*, 196. [CrossRef]
11. Abhadiomhen, S.E.; Nzeakor, E.O.; Oyibo, K. Health Risk Assessment Using Machine Learning: Systematic Review. *Electronics* **2024**, *13*, 4405. [CrossRef]
12. Laine, T.H.; Lindberg, R.S.N. Designing Engaging Games for Education: A Systematic Review on Game Motivators and Design Principles. *IEEE Trans. Learn. Technol.* **2020**, *13*, 804–821. [CrossRef]
13. Alshiha, M.B.; Al-Abdullatif, A.M. Gamification in Flipped Classrooms for Sustainable Digital Education: The Influence of Competitive and Cooperative Gamification on Learning Outcomes. *Sustainability* **2024**, *16*, 10734. [CrossRef]
14. Kim, J.; Castelli, D.M. Effects of Gamification on Behavioral Change in Education: A Meta-Analysis. *Int. J. Environ. Res. Public Health* **2021**, *18*, 3550. [CrossRef] [PubMed]
15. Barricelli, B.R.; Casiraghi, E.; Fogli, D. A Survey on Digital Twin: Definitions, Characteristics, Applications, and Design Implications. *IEEE Access* **2019**, *7*, 167653–167671. [CrossRef]
16. Ruiu, P.; Nitti, M.; Pilloni, V.; Cadoni, M.; Grosso, E.; Fadda, M. Metaverse & Human Digital Twin: Digital Identity, Biometrics, and Privacy in the Future Virtual Worlds. *Multimodal Technol. Interact.* **2024**, *8*, 48. [CrossRef]
17. Lu, M.; Hu, Z. Digital Twin-Enhanced Programming Education: An Empirical Study on Learning Engagement and Skill Acquisition. *Computers* **2025**, *14*, 322. [CrossRef]
18. Kritzinger, W.; Karner, M.; Traar, G.; Henjes, J.; Sihn, W. Digital Twin in manufacturing: A categorical literature review and classification. *IFAC-Pap.* **2018**, *51*, 1016–1022. [CrossRef]
19. Jones, D.; Snider, C.; Nassehi, A.; Yon, J.; Hicks, B. Characterising the Digital Twin: A systematic literature review. *CIRP J. Manuf. Sci. Technol.* **2020**, *29*, 36–52. [CrossRef]
20. Guajardo-Cuéllar, A.; Corona-Echauri, R.; Meza-Flores, R.A.; Vázquez, C.R.; Rodríguez-Arreola, A.; Navarro-Gutiérrez, M. Mixed Reality Laboratory for Teaching Control Concepts: Design, Validation, and Implementation. *Educ. Sci.* **2025**, *15*, 883. [CrossRef]
21. Hu, W.; Lei, Z.; Zhou, H.; Liu, G.-P.; Deng, Q.; Zhou, D.; Liu, Z.-W. Plug-in Free Web-Based 3-D Interactive Laboratory for Control Engineering Education. *IEEE Trans. Ind. Electron.* **2017**, *64*, 3808–3818. [CrossRef]
22. Zalozhnev, A.Y.; Ginz, V.N. Industry 4.0: Underlying technologies. Industry 5.0: Human–computer interaction as a tech bridge from Industry 4.0 to Industry 5.0. In Proceedings of the 2023 9th International Conference on Web Research (ICWR), Tehran, Iran, 3–4 May 2023; pp. 232–236.
23. Islam, M.T.; Sepanloo, K.; Woo, S.; Woo, S.H.; Son, Y.-J. A Review of the Industry 4.0 to 5.0 Transition: Exploring the Intersection, Challenges, and Opportunities of Technology and Human–Machine Collaboration. *Machines* **2025**, *13*, 267. [CrossRef]
24. Domínguez, L.G.I. Digital Twins in Industry 5.0—Systematic Literature Review. *Espacios (EPSIR)* **2024**, *9*, 641.
25. Folgado, F.J.; Calderón, D.; González, I.; Calderón, A.J. Review of Industry 4.0 from the Perspective of Automation and Supervision Systems: Definitions, Architectures and Recent Trends. *Electronics* **2024**, *13*, 782. [CrossRef]
26. Pacheco-Velázquez, A.; Ramírez-Noriega, A.; Ruiz-Ramírez, J.; Ramírez-Morales, I. Educational Simulators for Logistics Training in Industry 4.0 Contexts. *Front. Educ.* **2024**, *9*, 1331911.

27. Brandl, L.C.; Schrader, A. Realizing Ambient Serious Games in Higher Education—Concept and Heuristic Evaluation. *Trends High. Educ.* **2025**, *4*, 52. [CrossRef]
28. Žilak, M.; Car, Ž. A Framework for Improving Accessibility of Serious Games in Handheld Augmented Reality Based on User Interaction Data. *Appl. Sci.* **2025**, *15*, 2161. [CrossRef]
29. Bertozzi, G.; Paciarotti, C.; Schiraldi, M. Implementing Serious Games through a Pedagogical Lens in Engineering Education: An Experimental Study. *Eur. J. Eng. Educ.* **2024**, *49*, 1131–1157. [CrossRef]
30. Rodríguez-Calzada, L.; Paredes-Velasco, M.; Urquiza-Fuentes, J. The Educational Impact of a Comprehensive Serious Game within the University Setting: Improving Learning and Fostering Motivation. *Heliyon* **2024**, *10*, e35608. [CrossRef] [PubMed]
31. Gordillo, A.; López-Fernández, D.; Mayor, J. Examining and Comparing the Effectiveness of Virtual Reality Serious Games and LEGO Serious Play for Learning Scrum. *Appl. Sci.* **2024**, *14*, 830. [CrossRef]
32. Núñez Pacheco, R.; Espinoza Montoya, C.; Yucra Quispe, L.M.; Turpo Gebera, O.; Aguaded, I. Serious video games in engineering education: A scoping review. *J. Technol. Sci. Educ.* **2023**, *13*, 446–460. [CrossRef]
33. Sánchez, J.; Dormido-Canto, S.; Farías, G.; Godoy, F.; Dormido, S. Understanding automatic control concepts by playing games. *Int. J. Eng. Educ.* **2011**, *27*, 558–573.
34. Ortiz, J.S.; Quishpe, E.K.; Sailema, G.X.; Guamán, N.S. Digital Twin-Based Active Learning for Industrial Process Control and Supervision in Industry 4.0. *Sensors* **2025**, *25*, 2076. [CrossRef] [PubMed]
35. Guc, F.; Viola, J.; Chen, Y. Digital twins enabled remote laboratory learning experience for mechatronics education. In Proceedings of the 2021 IEEE 1st International Conference on Digital Twins and Parallel Intelligence (DTPI), Beijing, China, 15–20 July 2021; pp. 242–245.
36. Loizou, S.; Andreou, A.S. A Framework for Standardizing the Development of Serious Games with Real-Time Self-Adaptation Capabilities Using Digital Twins. *Technologies* **2025**, *13*, 369. [CrossRef]
37. Zhang, J.; Zhu, J.; Tu, W.; Wang, M.; Yang, Y.; Qian, F.; Xu, Y. The Effectiveness of a Digital Twin Learning System in Assisting Engineering Education Courses: A Case of Landscape Architecture. *Appl. Sci.* **2024**, *14*, 6484. [CrossRef]
38. Rajamäki, J. Digital Twin Technology Training and Research in Health Higher Education: A Review. *Explor. Digit. Health Technol.* **2024**, *2*, 188–201. [CrossRef]
39. Asciak, L.; Kyeremeh, J.; Luo, X.; Kazakidi, A.; Connolly, P.; Picard, F.; O'Neill, K.; Tsaftaris, S.A.; Stewart, G.D.; Shu, W.; et al. Digital twin-assisted surgery: Concept, opportunities, and challenges. *NPJ Digit. Med.* **2025**, *8*, 32. [CrossRef]
40. Cellina, M.; Cè, M.; Alì, M.; Irmici, G.; Ibba, S.; Caloro, E.; Fazzini, D.; Oliva, G.; Papa, S. Digital Twins: The New Frontier for Personalized Medicine? *Appl. Sci.* **2023**, *13*, 7940. [CrossRef]
41. Visioli, A. Teaching Control in the Era of Industry 4.0. *Front. Control Eng.* **2023**, *4*, 1228462.
42. Azofeifa, J.D.; Rueda-Castro, V.; Camacho-Zuñiga, C.; Chans, G.M.; Membrillo-Hernández, J.; Caratozzolo, P. Future Skills for Industry 4.0 Integration and Innovative Learning for Continuing Engineering Education. *Front. Educ.* **2024**, *9*, 1412018. [CrossRef]
43. Ahmad, I.; Sharma, S.; Singh, R.; Gehlot, A.; Gupta, L.R.; Thakur, A.K.; Priyadarshi, N.; Twala, B. Inclusive Learning Using Industry 4.0 Technologies: Addressing Student Diversity in Modern Education. *Cogent Educ.* **2024**, *11*, 2330235. [CrossRef]
44. Global PID Controllers Market Report. PRNewswire 2023. Available online: https://www.prnewswire.com/news-releases/global-pid-controllers-market-report-2023-industry-4-0-to-give-market-impetus{-}{-}forecast-to-2030{-}{-}302002422.html (accessed on 18 August 2025).
45. Dapkute, A.; Siozinys, V.; Jonaitis, M.; Kaminickas, M.; Siozinys, M. Enhancing Industrial Process Control: Integrating Intelligent Digital Twin Technology with Proportional-Integral-Derivative Regulators. *Machines* **2024**, *12*, 319. [CrossRef]
46. Chen, Y.-P.; Karkaria, V.; Tsai, Y.-K.; Rolark, F.; Quispe, D.; Gao, R.X.; Cao, J.; Chen, W. Real-time decision-making for Digital Twin in additive manufacturing with Model Predictive Control using time-series deep neural networks. *J. Manuf. Syst.* **2025**, *80*, 412–424. [CrossRef]
47. Brumar, R. OneAxisDrone-OpenRepository; GitHub Repository. Available online: https://github.com/AtthosTheGreat/OneAxisDrone-OpenRepository (accessed on 3 September 2025).
48. Yeo, I.-K. An algorithm for computing the exact distribution of the Wilcoxon signed-rank statistic. *J. Korean Stat. Soc.* **2017**, *46*, 328–338. [CrossRef]